

Now in this search box, instead of entering a username, we enter:

```
jsmith';UPDATE salaries SET salary=200000 WHERE  
userid='jsmith
```

What this bit of code will do is, to sneak in the second UPDATE SQL command into the original request, and have it execute on the server. This command updates the salary field of the salary table in the database, and sets it to 200,000 instead of 50,000, giving our friend “jsmith” a good 400% boost!

The form below allows a user to view salaries associated with a userid (from the table named **salaries**). This form is vulnerable to String SQL Injection. In order to pass this lesson, use SQL Injection to modify the salary for user **jsmith**.

Enter your userid:

USERID	SALARY
jsmith	50000

The original page.

The form below allows a user to view salaries associated with a userid (from the table named **salaries**). This form is vulnerable to String SQL Injection. In order to pass this lesson, use SQL Injection to modify the salary for user **jsmith**.

Enter your userid:

USERID	SALARY
jsmith	200000

The way to protect against

such attacks is to use a parameterized interface or an application framework API for the database. Each parameter used should be sanitized depending on its type. For example, in the first case we expect a number for the userid, so the parameter should be converted to an integer in the backend, discarding the rest of the string. Strings should be escaped.

We hope that you are now very clear about how an injection attack works, and how important it is to prevent it. With a little though put in while developing an application it is quite easy to avoid injection attacks.

3.2.3 Cross-Site Scripting

“XSS flaws occur whenever an application takes untrusted data and sends it to a web browser without proper validation and escaping. XSS allows attackers to execute scripts in the victim’s browser which can hijack user sessions, deface web sites, or redirect the user to malicious sites.”

Much like the injection vulnerability we explored previously, XSS flaws occur due to improper sanitization of untrusted data. Untrusted data here is any kind of data which does not originate from the server, but is provided in the URL and can be modified by the end-user / attacker.

Unlike the injection vulnerabilities, where the data being run by an interpreter on the server end, an XSS attack it targeted towards the users